



3.3

Queries using Group by, Having and Order by clause

Prepared By
Mohammad Ali
Lecturer (Selection Grade)
M. H. Saboo Siddik Polytechnic, Mumbai

Data Query Language (DQL) – SELECT

SQL SELECT Statement:

- DQL is used to fetch the data from the database.
- DQL commands are basically SELECT statements.
- SELECT statements let us query the database to find information in one or more tables, and return the query as a result set.
- SELECT is used to select the attribute based on the condition described by WHERE clause.
- Basic syntax of SELECT Command:
SELECT *column1, column2, ...*
FROM *table_name;*

Syntax of SQL SELECT Statement:

- SELECT** *column_list*
FROM *table-name*
[**WHERE** Clause]
[**GROUP** BY clause]
[**HAVING** clause]
[**ORDER** BY clause];
- *table-name* is the name of the table from which the information is retrieved.
 - *column_list* includes one or more columns from which data is retrieved.
 - The code within the brackets is optional

Data Query Language (DQL) – SELECT

Constants and Arithmetic

As well as column names, we can select

- constants,
- compute arithmetic expressions and
- evaluate functions

in a **SELECT** statement

EXAMPLES:

```
SELECT Mark/100 FROM Grades;
```

```
SELECT sal + comm FROM emp;
```

```
SELECT 1.175*Price FROM Products;
```

```
SELECT empno, sal, sal+500,  
sal+comm FROM emp where comm  
is not null;
```

```
SQL> SELECT empno, sal, sal+500, sal+comm  
2 FROM emp where comm is not null;
```

EMPNO	SAL	SAL+500	SAL+COMM
7499	1600	2100	1900
7521	1250	1750	1750
7654	1250	1750	2650
7844	1500	2000	1500

ORDER BY

- The **ORDER BY** clause sorts the results of a query
 - We can sort in ascending (default) or descending order
 - Multiple columns can be given
 - We cannot order by a column which isn't in the result.

```
SELECT <columns>
FROM <tables>
WHERE <condition>
ORDER BY <columns>
[ASCENDING |
DESCENDING |
ASC | DESC ]
```

ORDER BY Example

```
select * from dept;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

```
select * from dept ORDER  
BY dname;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
40	OPERATIONS	BOSTON
20	RESEARCH	DALLAS
30	SALES	CHICAGO

ORDER BY Example

```
CREATE TABLE GRADES
(SNAME VARCHAR2(10),
C_CODE VARCHAR2(3),
MARKS NUMBER(3));
```

```
SQL> CREATE TABLE GRADES
2 (SNAME VARCHAR2(10),
3 C_CODE VARCHAR2(3),
4 MARKS NUMBER(3));
Table created.
```

*D.SQL - Notepad

File Edit Format View Help

INSERT ALL

```
INTO GRADES VALUES ('ANAM', 'DMS', 68)
INTO GRADES VALUES ('ANAM', 'CGR', 64)
INTO GRADES VALUES ('JUNAID', 'DMS', 66)
INTO GRADES VALUES ('JUNAID', 'CGR', 67)
INTO GRADES VALUES ('HABBAN', 'DMS', 70)
INTO GRADES VALUES ('HABBAN', 'CGR', 65)
INTO GRADES VALUES ('CHAITANYA', 'DMS', 60)
INTO GRADES VALUES ('CHAITANYA', 'CGR', 62)
SELECT * FROM DUAL;
```

Ln 10, Col 100% Windows (CRLF) UTF-8

```
SQL> ED C:\NOTES1\D.SQL
SQL> @ C:\NOTES1\D.SQL
8 rows created.
```

SELECT * FROM Grades;

```
SQL> SELECT * FROM GRADES;
```

SNAME	C_C	MARKS
ANAM	DMS	68
ANAM	CGR	64
JUNAID	DMS	66
JUNAID	CGR	67
HABBAN	DMS	70
HABBAN	CGR	65
CHAITANYA	DMS	60
CHAITANYA	CGR	62

8 rows selected.

SNAME	C_CODE	MARKS
ANAM	DMS	68
ANAM	CGR	64
JUNAID	DMS	66
JUNAID	CGR	67
HABBAN	DMS	70
HABBAN	CGR	65
CHAITANYA	DMS	60
CHAITANYA	CGR	62

SELECT * FROM Grades ORDER BY C_Code ASC, Marks DESC;

```
SQL> SELECT * FROM Grades
2 ORDER BY C_Code ASC,
3 Marks DESC;
```

SNAME	C_C	MARKS
JUNAID	CGR	67
HABBAN	CGR	65
ANAM	CGR	64
CHAITANYA	CGR	62
HABBAN	DMS	70
ANAM	DMS	68
JUNAID	DMS	66
CHAITANYA	DMS	60

8 rows selected.

SNAME	C_CODE	MARKS
JUNAID	CGR	67
HABBAN	CGR	65
ANAM	CGR	64
CHAITANYA	CGR	62
HABBAN	DMS	70
ANAM	DMS	68
JUNAID	DMS	66
CHAITANYA	DMS	60

GROUP BY

INTRODUCTION

- GROUP BY Clause is one of the tools to summarize or aggregate the data series.
- For example,
 - To count the no. of employees in each department of the company.
 - To find the total, average, minimum and maximum salary of each department.
 - To find the average mark of each student.
- It groups the databases on the basis of one or more column and aggregates the results.
- Hence, we want to apply aggregate functions to groups of rows

INTRODUCTION

- The GROUP BY Clause is utilized in SQL with the SELECT statement to organize similar data into groups.
- It combines the multiple records in single or more columns using aggregate functions.
- GROUP BY aggregates the results on the basis of selected column: COUNT, MAX, MIN, SUM, AVG, etc.
- GROUP BY returns only one result per group of data.
- GROUP BY Clause always follows the WHERE Clause.
- GROUP BY Clause always precedes the ORDER BY.

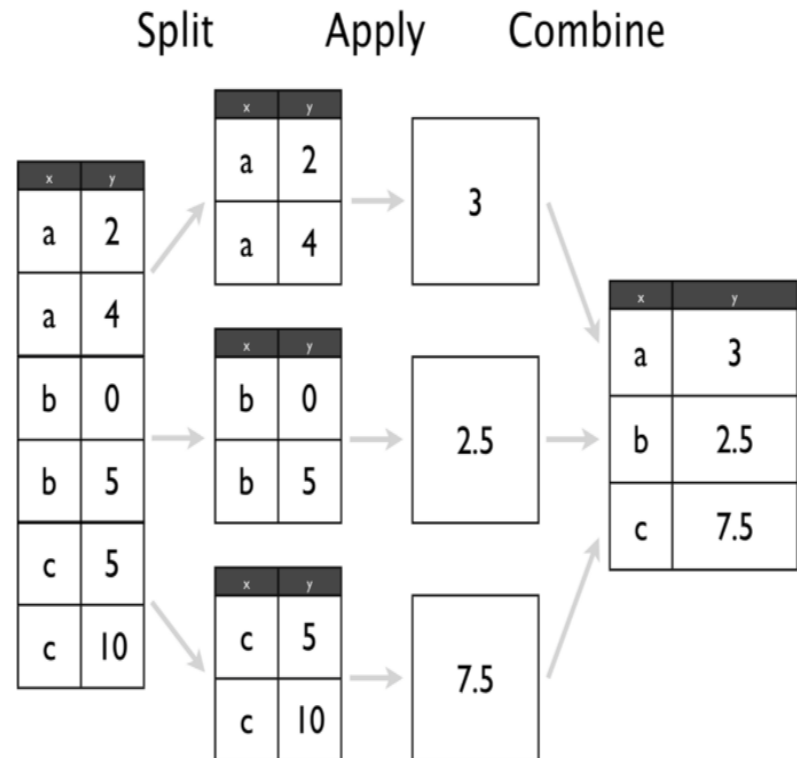
GROUP BY

SYNTAX

```
SELECT <columns>, <aggregate_function(column)>
FROM <tables>
WHERE <condition>
GROUP BY <columns>
```

- It uses the **split-apply-combine** strategy for data analysis.
- In the **split phase**, It divides the groups with its values.
- In the **apply phase**, It applies the aggregate function and generates a single value.
- In the **combine phase**, It combines the groups with single values into a single value

SPLIT-APPLY-COMBINE Strategy



GROUP BY EXAMPLE

SNAME	C_CODE	MARKS
ANAM	DMS	68
ANAM	CGR	64
JUNAID	DMS	66
JUNAID	CGR	67
HABBAN	DMS	70
HABBAN	CGR	65
CHATANYA	DMS	60
CHATANYA	CGR	62

SELECT SNAME, SUM(MARKS)
FROM GRADES
GROUP BY SNAME;

GROUP BY
GRADES Table
using SNAME

```
SQL> SELECT SNAME, SUM(MARKS)
2 FROM GRADES
3 GROUP BY SNAME;
```

SNAME	SUM(MARKS)
-----	-----
CHAITANYA	122
JUNAID	133
HABBAN	135
ANAM	132

In above example, Table is grouped based on the SNAME column and MARKS is aggregated Student-wise.

GROUP BY
GRADES Table
using C_CODE

SELECT C_CODE, MAX(MARKS),
MIN(MARKS), AVG(MARKS)
FROM GRADES
GROUP BY C_CODE;

```
SQL> SELECT C_CODE, MAX(MARKS),
2 MIN(MARKS), AVG(MARKS)
3 FROM GRADES
4 GROUP BY C_CODE;
```

C_C	MAX(MARKS)	MIN(MARKS)	AVG(MARKS)
---	-----	-----	-----
CGR	67	62	64.5
DMS	70	60	66

In above example, Table is grouped based on the C_CODE column and MARKS is aggregated Course-wise.

GROUP BY EXAMPLE

```
SELECT deptno, job, COUNT(*) "EMPLOYEE COUNT",  
       AVG(sal), SUM(sal) FROM emp  
GROUP BY deptno, job ORDER BY deptno, job;
```

```
SQL> SELECT deptno, job, COUNT(*) "EMPLOYEE COUNT", AVG(sal), SUM(sal)  
2 FROM emp GROUP BY deptno, job ORDER BY deptno, job;
```

DEPTNO	JOB	EMPLOYEE COUNT	AVG(SAL)	SUM(SAL)
10	CLERK	1	1300	1300
10	MANAGER	1	2450	2450
10	PRESIDENT	1	5000	5000
20	ANALYST	2	3000	6000
20	CLERK	2	950	1900
20	MANAGER	1	2975	2975
30	CLERK	1	950	950
30	MANAGER	1	2850	2850
30	SALESMAN	4	1400	5600

9 rows selected.

```
SELECT deptno, job, COUNT(*) "EMPLOYEE COUNT",  
       AVG(sal), SUM(sal) FROM emp  
GROUP BY deptno, job ORDER BY job, deptno;
```

```
SQL> SELECT deptno, job, COUNT(*) "EMPLOYEE COUNT",  
2 AVG(sal), SUM(sal) FROM emp  
3 GROUP BY deptno, job ORDER BY job, deptno;
```

DEPTNO	JOB	EMPLOYEE COUNT	AVG(SAL)	SUM(SAL)
20	ANALYST	2	3000	6000
10	CLERK	1	1300	1300
20	CLERK	2	950	1900
30	CLERK	1	950	950
10	MANAGER	1	2450	2450
20	MANAGER	1	2975	2975
30	MANAGER	1	2850	2850
10	PRESIDENT	1	5000	5000
30	SALESMAN	4	1400	5600

9 rows selected.

The more columns included in the **GROUP BY** clause, the more granular the aggregates. The following example groups by the **DEPTNO** and **JOB** columns of EMP Table

Remember, all non-aggregate columns must be included in the GROUP BY clause.

HAVING Clause

INTRODUCTION

- HAVING Clause used in SQL as a conditional Clause with GROUP BY Clause.
- This conditional clause returns rows where aggregate function results matched with given conditions only.
- It added in the SQL because WHERE Clause cannot be combined with aggregate results.
- HAVING Clause always utilized in combination with GROUP BY Clause.
- HAVING Clause restricts the data on the group records rather than individual records.
- WHERE and HAVING can be used in a single query.

SYNTAX

```
SELECT <columns>,  
        <aggregate_function(column)>  
FROM <tables>  
WHERE <condition>  
GROUP BY <columns>  
HAVING <condition>
```

HAVING Clause EXAMPLE

SNAME	C_CODE	MARKS
ANAM	DMS	68
ANAM	OGR	64
JUNAID	DMS	66
JUNAID	OGR	67
HABBAN	DMS	70
HABBAN	OGR	65
CHATANYA	DMS	60
CHATANYA	OGR	62

```
SELECT SNAME, SUM(MARKS)
FROM GRADES
GROUP BY SNAME
HAVING SUM(MARKS)>132;
```

GROUP BY
GRADES Table
using SNAME

```
SQL> SELECT SNAME, SUM(MARKS)
2 FROM GRADES
3 GROUP BY SNAME
4 HAVING SUM(MARKS)>132;
```

SNAME	SUM(MARKS)
JUNAID	133
HABBAN	135

In above example, Table is grouped based on the SNAME column and these grouped rows filtered using HAVING Clause with condition SUM(MARKS) > 132.

WHERE and HAVING clause

Sr. No.	WHERE CLAUSE	GROUP BY CLAUSE
1	The WHERE clause specifies the criteria which individual records must meet to be selected by a query. It can be used without the GROUP BY clause.	The HAVING clause can not be used without the GROUP BY clause.
2	The WHERE clause selects rows before grouping.	The HAVING clause selects rows after grouping.
3	WHERE clause refers to the rows of tables, and so cannot use AGGREGATE FUNCTIONS.	HAVING clause refers to the groups of rows, and so cannot use columns which are not in the GROUP BY and it can contain AGGREGATE FUNCTIONS.
4	WHERE clause is used to impose condition on SELECT statement as well as single row function and is used before GROUP BY clause.	HAVING clause is used to impose condition on multi row functions(AGGREGATE Functions) and is used after GROUP BY clause.
5	SELECT job, COUNT(*) "EMPLOYEE COUNT" FROM emp WHERE job in ('CLERK','MANAGER','SALESMAN') GROUP BY job HAVING COUNT(*)>3;	

```
SQL> SELECT job, COUNT(*) "EMPLOYEE COUNT"  
2  FROM emp WHERE job in ('CLERK','MANAGER','SALESMAN')  
3  GROUP BY job HAVING COUNT(*)>3;
```

```
JOB      | EMPLOYEE COUNT  
----- | -----  
CLERK    |              4  
SALESMAN |              4
```